ISA5810 DM2026 Lab 1 — Master Notebook Questions

Fill in this file as you work through DM2026-Lab1-Master.ipynb. Submit this completed file converted to PDF together with the rest of your Lab 1 work.

Please do not change the style of this document.

Question numbers 1–10 follow the same order as the notebook (top to bottom).

Student Information

Please complete the fields below. For **English Name** and **Chinese Name**, you may fill in **only one** if you do not have the other.

- **English Name: Wei-Fu Chen**
- **Chinese Name: 陳蔚榑**
- **Email: chenweifu9999@gmail.com**
- **Student ID: 115062806**
- **University: NTHU**

Instructions

These questions go with the Master Jupyter notebook (DM2026-Lab1-Master.ipynb). Each question is tied to a specific notebook section. Open the notebook, find **Question N (take home)**, and write your answer here.

We **strongly** suggest running the notebook to learn the lab, but the TAs will **not** audit whether you executed every cell to answer the questions in this document.

How to answer each question

1. Find the **notebook section** listed under the question (use the Table of Contents if needed).
2. Read the task; change parameters when asked (e.g. stop_words, thresholds, k, minSup).
3. Write a clear answer in the **Answer** space below each question.
4. Leave the question text unchanged; write only in the answer and disclosure fields.

Sources, honesty, and AI use

Acceptable vs. Unacceptable AI Usage: Generative AI should serve as an interactive tutor to guide your understanding, not as an automated solver completing the assignment for you.

- **Permitted (AI as Guide & Editor):**
  - Asking AI to explain unfamiliar theoretical concepts, discuss algorithmic trade-offs, or troubleshoot runtime errors in the notebook.
  - Polishing the grammar, phrasing, and readability of your own original, self-written drafts.

- **Prohibited (AI as Solver):**
  - Copying assignment questions directly into an AI tool to generate answers.
  - Using AI to fabricate experimental results or observations without executing the code in DM2026-Lab1-Master.ipynb.
  - Submitting machine-generated prose wholesale.

- **External material:** If you use websites, books, papers, or documentation beyond the notebook, cite them with **clickable links** (or DOI/ISBN when no URL exists).

- **When AI was used:** Your answer is acceptable if you (1) **disclose** AI use and (2) include **at least one external link** that validates the information in your answer (official docs, textbook, paper, or reputable tutorial — not the AI chat alone as the only source).

- **When AI was not used:** You may write "No" under **AI used** or leave it blank only if you truly did not use AI for that question. References are still encouraged if you looked anything up.

Under each question, complete:

- **AI used for this answer:** Yes / No
- **If yes, how:** (e.g. brainstorming, grammar, concept explanation)
- **External references (links):** *(required when AI = Yes; encouraged otherwise)*

Write in English. Keep answers concise and specific.

Questions

Question 1: Data Filtering & Domain Relevance

Notebook: Section 3.2 — Question 1 (take home)

Task: Filter the dataframe **X** for the category **sci.med** and check its length.

Questions:

1. How many documents are in this specific category?
2. From a data mining perspective, why is isolating specific categories a crucial first step before applying algorithms like frequent pattern mining?

Answer: 1. There are 594 documents in the sci.med category. 2. Filtering the medical category helps identify word combinations that frequently occur within medical documents. Without this filtering, patterns from unrelated categories may dominate the results and make medical patterns harder to identify.

AI used for this answer: Yes / No

If yes, how: Concept clarification and English wording revision based on my own explanation.

External references (links): https://rasbt.github.io/mlxtend/user_guide/frequent_patterns/fpgrowth/

Question 2: Handling Missing Data

Notebook: Section 4.1 — Question 2 (take home) (after the **dropna()** demonstration)

Task: In Section 4.1, a dummy record with a missing value is injected and then removed using the **dropna()** function.

Question: If 40% of documents were missing the **category_name** label, why would simply dropping these rows (with **dropna()**) be a flawed data mining strategy? Name one objective alternative so the text data can still be utilized.

Answer: Dropping these rows would discard 40% of the documents, even though their text may still be useful. It could also introduce bias if missing labels are concentrated in particular categories. An alternative is to retain the unlabeled documents for unsupervised analysis, such as clustering or frequent pattern mining, without assuming their category labels.

AI used for this answer: Yes / No

If yes, how: Concept clarification and English wording revision based on my own explanation.

External references (links): https://scikit-learn.org/stable/modules/clustering.html

---

Question 3: Data Sampling Risks

Notebook: Section 5.1 — Question 3 (take home)

Task: In Section 5.1, the code uses **X_sample = X.sample(n=1000)** to create a subset. Compare the bar chart of the original category distribution to the sampled distribution.

Questions:

1. Do the exact category proportions perfectly match the original?
2. What is the specific risk of simple random sampling on a highly imbalanced dataset?

Answer: 1. No, the sampled category proportions do not exactly match the original proportions. The bar charts show differences in the relative proportions of the four categories because of random sampling variation. 2. In a highly imbalanced dataset, simple random sampling may select very few documents from minority classes or miss them entirely. This can make the sample less representative and make patterns in minority classes harder to identify.

AI used for this answer: Yes / No

If yes, how: Concept clarification, assistance with checking category proportions, and English wording revision based on my own explanation and observations.

External references (links): https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.StratifiedShuffleSplit.html

---

Question 4: Text Vectorization and Matrix Sparsity

Notebook: Section 5.3 — Question 4 (take home)

Task: In Section 5.3, the **CountVectorizer** converts text into a document-term matrix using **stop_words='english'**. Change this parameter to **stop_words=None** and re-run the cell to check the shape of **X_counts**.

Questions:

1. Does the total number of features (columns) increase or decrease?
2. Since stopwords are highly common words that appear in almost all documents, what effect does adding them back have on the overall sparsity (the percentage of zeros) of the matrix?

Answer: 1. The number of features increased from 35,482 to 35,788 because setting stop_words=None retains stopwords that were previously removed. 2. Stopwords occur in many documents, so their added columns contain relatively more non-zero values. Adding these denser columns is therefore expected to decrease the overall sparsity, meaning a lower percentage of zero entries.

AI used for this answer: Yes / No

If yes, how: Concept clarification and English wording revision based on my own explanation and experimental results.

External references (links): https://scikit-learn.org/stable/modules/feature_extraction.html#text-feature-extraction

---

Question 5: Feature Selection — Variance Thresholding

Notebook: Section 5.3.1 — Question 5 (take home)

Task: In the variance filtering section, re-run **variance_filter** using threshold values **threshold = 0.0, threshold = 0.01, and threshold = 0.05**. Record the number of features marked as **keep == True**.

Questions:

1. What happens to the total size of the vocabulary as the variance threshold increases from 0.0 to 0.05?
2. In terms of document frequency, what kinds of words have variance close to zero and are eliminated first?
3. Why is eliminating near-zero variance features considered a safe way to reduce dimensionality without harming model accuracy?

Answer:

1. As the variance threshold increased, the retained vocabulary became smaller. The numbers of terms kept were:

Threshold	Number of terms kept (keep == True)
0.0	35482
0.01	5089
0.05	1491

2. Words that occur rarely with low counts, or have nearly constant counts across documents, tend to have low variance and are removed first. High document frequency alone does not imply low variance if the counts vary substantially. 3. Removing near-zero variance features reduces dimensionality and computational cost because these features usually provide little information for distinguishing documents. However, rare terms may still be useful for minority classes, so unchanged accuracy is not guaranteed and should be checked through validation.

AI used for this answer: Yes / No

If yes, how: Concept clarification and English wording revision based on my own explanation and experimental results.

External references (links): https://scikit-learn.org/stable/modules/feature_selection.html#removing-features-with-low-variance

---

Question 6: Feature Correlation — Linear (Pearson) vs. Monotonic (Spearman)

Notebook: Section 5.3.2 — Question 6 (take home)

Task: Run the correlation filter against the binary indicator for the **comp.graphics** class. Inspect the top 10 features sorted by **pearson_r** and compare them to the top 10 features sorted by **spearman_r**.

Questions:

1. Are the top-correlated words generic stop words or domain-specific terminology for graphics?
2. What is the mathematical difference in what **Pearson** correlation measures versus what **Spearman** rank correlation measures?
3. In what specific scenario would a word have a high **Spearman** rank correlation with a topic, but a significantly lower **Pearson** correlation?

Answer: 1. Pearson's top terms include general expressions and header-related terms, such as thanks and keywords, alongside graphics-related terms such as vga and video. Spearman's top terms contain more graphics-related vocabulary, including graphics, files, image, and 3d. 2. Pearson correlation measures the linear relationship between raw term counts and the binary class indicator. Spearman correlation measures the monotonic relationship using ranks rather than raw values. 3. A word may consistently occur more frequently in graphics documents than in other categories, while a few documents repeat it extremely often. These extreme counts can weaken Pearson correlation, whereas Spearman may remain high because it uses ranks. In my results, graphics had Spearman correlation 0.5211 and Pearson correlation 0.1779, although these coefficients alone do not establish the cause of the difference.

AI used for this answer: Yes / No

If yes, how: Concept clarification and English wording revision based on my own explanation and experimental results.

External references (links):
https://docs.scipy.org/doc/scipy/reference/generated/scipy.stats.pearsonr.html
https://docs.scipy.org/doc/scipy/reference/generated/scipy.stats.spearmanr.html

---

Question 7: Frequent Pattern Mining — Algorithm & Parameter Dynamics

Notebook: Section 5.4.2 — Question 7 (take home)

Task: In the Frequent Pattern Mining section, run **Top-K FAE** with **k = 100 and k = 500**. Then, run **MaxFPGrowth** using minimum support thresholds **minSup=9 and minSup= 3**.

Questions:

1. How does decreasing **minSup** (from 9 to 3) in **MaxFPGrowth** affect the execution runtime and the total number of discovered patterns?
2. How does the parameter k in **Top-K FAE** differ fundamentally from **minSup** in **MaxFPGrowth** when deciding which patterns to keep?
3. Why are "maximal" frequent patterns structurally more concise than standard frequent patterns?

Answer (1) — Runtime and pattern count when lowering minSup: In my experiment on alt.atheism, lowering minSup from 9 to 3 increased the number of maximal frequent patterns from 710 to 9,547. Runtime increased from approximately 0.0790 to 1.6487 seconds. The lower threshold allowed more patterns to qualify as frequent and expanded the search.

Answer (2) — Top-K k vs MaxFPGrowth minSup: In Top-K FAE, k specifies how many high-support patterns to return. My runs returned 100 and 500 patterns for k=100 and k=500, respectively. In MaxFPGrowth, minSup specifies the minimum transaction count required for a pattern to be frequent; it does not fix the number of results.

Answer (3) — Maximal vs standard frequent patterns: A maximal frequent pattern has no frequent proper superset. Therefore, maximal pattern mining omits frequent subsets already covered by larger frequent patterns, producing a more concise representation than listing every frequent pattern.

AI used for this answer: Yes / No

If yes, how: Troubleshooting file encoding, clarifying algorithm definitions, and revising English based on my own explanation and experimental results.

External references (links): https://udaylab.github.io/PAMI/html/frequentPatternMaximalmaxFPGrowth.html

---

Question 8: Dimensionality Reduction — Motivation

Notebook: Section 5.5 — Question 8 (take home) (before PCA/t-SNE code)

Task: After text vectorization, the **X_counts** matrix creates tens of thousands of distinct features (words).

Questions:

1. If we want to visually map how these documents group by category on a scatter plot, why is it mathematically and visually impossible to plot the raw **X_counts** matrix directly?
2. What specific category of data mining algorithms (referenced in the helper files) must be applied first?

Answer: 1. Each word represents a dimension, so each document is a point in a space with tens of thousands of dimensions. A standard 2D or 3D scatter plot cannot directly display all these dimensions. 2. Dimensionality reduction algorithms, such as PCA, t-SNE, or UMAP, transform the data into two or three dimensions so that we can visualize document distributions. However, the projection does not preserve all information from the original space.

AI used for this answer: Yes / No

If yes, how: Concept clarification and English wording revision based on my own explanation.

External references (links): https://scikit-learn.org/stable/modules/unsupervised_reduction.html

---

Question 9: Dimensionality Reduction — Linear (PCA) vs. Non-linear Manifolds (t-SNE/UMAP)

Notebook: Section 5.5 — Question 9 (take home) (after 2D/3D plots)

Task: In the 2D/3D Dimensionality Reduction section, compare the scatter plot projection generated by **PCA** against that generated by **t-SNE** or **UMAP**.

Questions:

1. Which method **(PCA or t-SNE/UMAP**) shows distinct, separated cluster islands for the 4 document categories, and which shows heavy overlap around the center?
2. Why does **PCA** struggle to separate high-dimensional text categories compared to **t-SNE** or **UMAP**?
3. Why must data analysts be careful not to over-interpret the physical distance between separate clusters in a **t-SNE** plot?

Answer: 1. In my plots, PCA shows heavy overlap near the center, with a few outliers. t-SNE and UMAP reveal more noticeable local groups, but neither completely separates the four document categories. 2. PCA finds linear directions that maximize overall variance, which does not necessarily correspond to category differences. With raw word counts, document length and extreme counts can influence these directions. Nonlinear methods such as t-SNE and UMAP can reveal local structures that a linear projection may miss. 3. t-SNE primarily preserves local neighborhood relationships rather than global distances. Therefore, the distance between separate clusters in the plot should not be interpreted as an accurate measure of their dissimilarity in the original space.

AI used for this answer: Yes / No

If yes, how: Concept clarification and English wording revision based on my own interpretation of the experimental plots.

External references (links): https://scikit-learn.org/stable/modules/manifold.html#t-distributed-stochastic-neighbor-embedding-t-sne

---

Question 10: One-Hot Binarization

Notebook: Section 5.6 — Question 10 (take home)

Task: In Section 3.1, examine **X['category']** (integers 0, 1, 2, 3). Then look at the output of **LabelBinarizer** in Section 5.6, where each category is converted into a binary vector (e.g., [1, 0, 0, 0]).

Questions:

1. How many binary columns are created to represent the 4 categories?
2. If an algorithm (like linear regression or KNN) is given the raw integer column **(0, 1, 2, 3)** as input, what false mathematical assumption does it make about the categories?
3. How does one-hot binarization eliminate this false assumption?

Answer: 1. Four binary columns are created, one for each category. Each document has one value of 1 indicating its category and three values of 0. 2. Treating the integer codes 0, 1, 2, and 3 as numerical inputs introduces an artificial ordering and numerical distances between categories. For example, category 0 would appear closer to category 1 than to category 3, although these codes are only identifiers. 3. One-hot encoding represents each category with a separate binary indicator. This removes the artificial ordering and distances caused by the arbitrary integer codes.

AI used for this answer: Yes / No

If yes, how: Concept clarification and English wording revision based on my own explanation and observed output.

External references (links): https://scikit-learn.org/stable/modules/preprocessing.html#encoding-categorical-features

---